- **Activity 3.1: Define FastAPI Routes**
    - Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions.

```
@app.post("/plan-home-budget")
async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_home_budget"]
= {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }
```

```
    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

```
@app.post("/generate-party")
async def plan_party_budget(
    budget_input: PartyBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate party budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_party_budget"] =
{
```

```
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
        "party_type": budget_input.party_type,
            "guests": budget_input.num_guests
          }

      # Get recommendations
result = get_party_recommendations(budget_input)

      # Save to history
      save_to_history(
    username=current_user.username,
    recommendation_type="party",
        input_data=budget_input.dict(),
        result=result
      )
    return result
```

Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```
@app.post("/generate-jewelry")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
```

```
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
```

```
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result

@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
```

```
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
return {"history": []}

    # Sort history by timestamp (newest first)
    history = sorted(
        user_recommendations[current_user.username],
        key=lambda x: x.timestamp,
        reverse=True
    )

    # Convert to dict for JSON response
    history_data = []
    for item in history:
        history_data.append({
            "id": item.id,
            "timestamp": item.timestamp,
            "type": item.recommendation_type,
            "input": item.input_summary,
            "summary": item.result_summary
        })

    return {"history": history_data}
```

- User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.

- Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.